

sw-rev-smoke

An AgentMPI run analysis

Abstract

Despite the name, this is not a smoke test of the software harness in general: it is the verification run for one specific bug fix, and the commit that fixed the bug is the commit that added this result file to the repository. In `real-sw-p8-full` the cross-review phase returned every critique over the *forward* review graph, so all sixteen reviews were delivered to ranks whose code the sender had not read and the phase was a silent no-op. This run, six ranks under the deterministic surrogate executor, drives the corrected code, and the trace settles the question: the twelve outgoing code messages travel on edges $r \rightarrow \{r+1, r+2\}$ and the twelve returning review messages on $r \rightarrow \{r-1, r-2\}$, an exact transpose, so **every review reached the rank that wrote the code it is about**. That is the answer to the routing question, and it is the opposite of the $p = 8$ finding. The run’s second lesson is about instrumentation. Both neighbourhood exchanges report `messages_sent: 0` — the payload of a `coll.neighbor_allgather` event of this vintage carries only `indegree`, `outdegree`, `label` and a wall time — so the trace viewer’s collectives panel shows MESSAGES 0 for the twelve invocations that in fact carried 24 of the run’s 95 messages. The whole justification of a neighbourhood collective is that it costs $\Theta(\text{degree})$ rather than $\Theta(p)$: at $p = 6$ it sent 12 messages where a ring allgather would have sent 30. The one collective in the library whose case rests entirely on its message count was the one collective not reporting it. Both facts have to be read through the executor: this run is `function`, not `broker`, so it says nothing whatever about agent behaviour.

1 What this run is

property	value
run	sw-rev-smoke
experiment	software
campaign label	sw-rev-smoke
declared world size	6
ranks appearing in the log	6
executors	<code>function</code> $\times$ 6
events	442
wall time	0.32 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.00×	total busy time over wall time
parallel efficiency	0.0%	achieved parallelism per rank
idle fraction	100.0%	rank-seconds paid for and unused
peak ranks busy	0	of 6 in the world
serial fraction of active time	0.0%	time with exactly one rank busy
load imbalance	0.00×	slowest rank’s busy time over the mean
coordination share	72.5%	rank-seconds blocked in collectives, of those available
coordination in flight	91.9%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	30	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	95	85 eager, 10 rendezvous
tokens sent	12,720	636 deferred under rendezvous
tokens in / out	45,501 / 846	prompt and completion
cost	\$0.1492	0.1764 \$/kTok out

3 What the analysis notices

- **[warning]** `neighbor_allgather/?` reports 0 messages but the fabric logged 12: the collective’s own accounting disagrees with the traffic it produced.
- **[warning]** `neighbor_allgather/?` reports 0 messages but the fabric logged 12: the collective’s own accounting disagrees with the traffic it produced.
- **[note]** Collectives account for 72.5% of the run’s rank-seconds, so more of the pool’s time went to coordination than to work.
- **[ok]** All 11 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	0.0	0	7	25	16	7,216	0.0	finalized
1	0.00	0.0	0	7	12	15	308	0.0	finalized
2	0.00	0.0	0	4	17	17	2,412	0.0	finalized
3	0.00	0.0	0	4	12	15	186	0.0	finalized
4	0.00	0.0	0	4	17	17	2,412	0.0	finalized
5	0.00	0.0	0	4	12	15	186	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank’s own lifetime, so a rank that joined late is not penalised for the time before it existed.

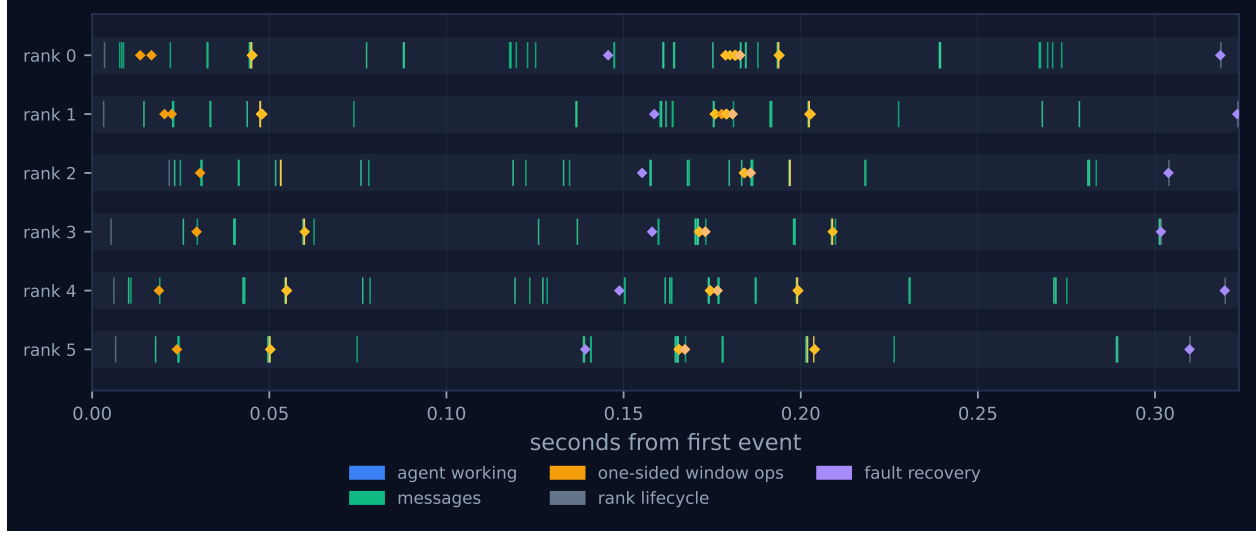


Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer’s palette so the same shapes read the same way in both.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens
bcast	binomial	spec	6	6	3	3	5	5	7,63
barrier	dissemination	win_fence:interfaces-0:interfaces-published	6	6	0	3	18	18	1
barrier	central	integrate-r1	6	6	0	2	0	0	
gather	binomial	collect-r1	6	6	3	3	5	5	28
bcast	binomial	report-r1	6	6	3	3	5	5	73
scatter	binomial	blame-r1	6	6	3	3	5	5	1,06
neighbor_allgather	?	review-r1	6	6	0	—	12	—	
neighbor_allgather	?	reviewback-r1	6	6	0	—	12	—	
barrier	dissemination	win_fence:interfaces-0:renegotiate-r1	6	6	0	3	18	18	1
barrier	central	integrate-r2	6	6	0	2	0	0	
gather	binomial	collect-r2	6	6	3	3	5	5	28
bcast	binomial	report-r2	6	6	3	3	5	5	73
scatter	binomial	blame-r2	6	6	3	3	5	5	1,06

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 looks nothing like the four-band bulk-synchronous picture the real-agent software runs give, and the reason is the executor. With `-executor function` every `agent.call` has `latency_s: 0.0`, so there are no work bars at all: the run’s 30 agent calls are instants, the entire 0.324s is protocol, and all six lanes are a dense stipple of message ticks, amber window diamonds and violet

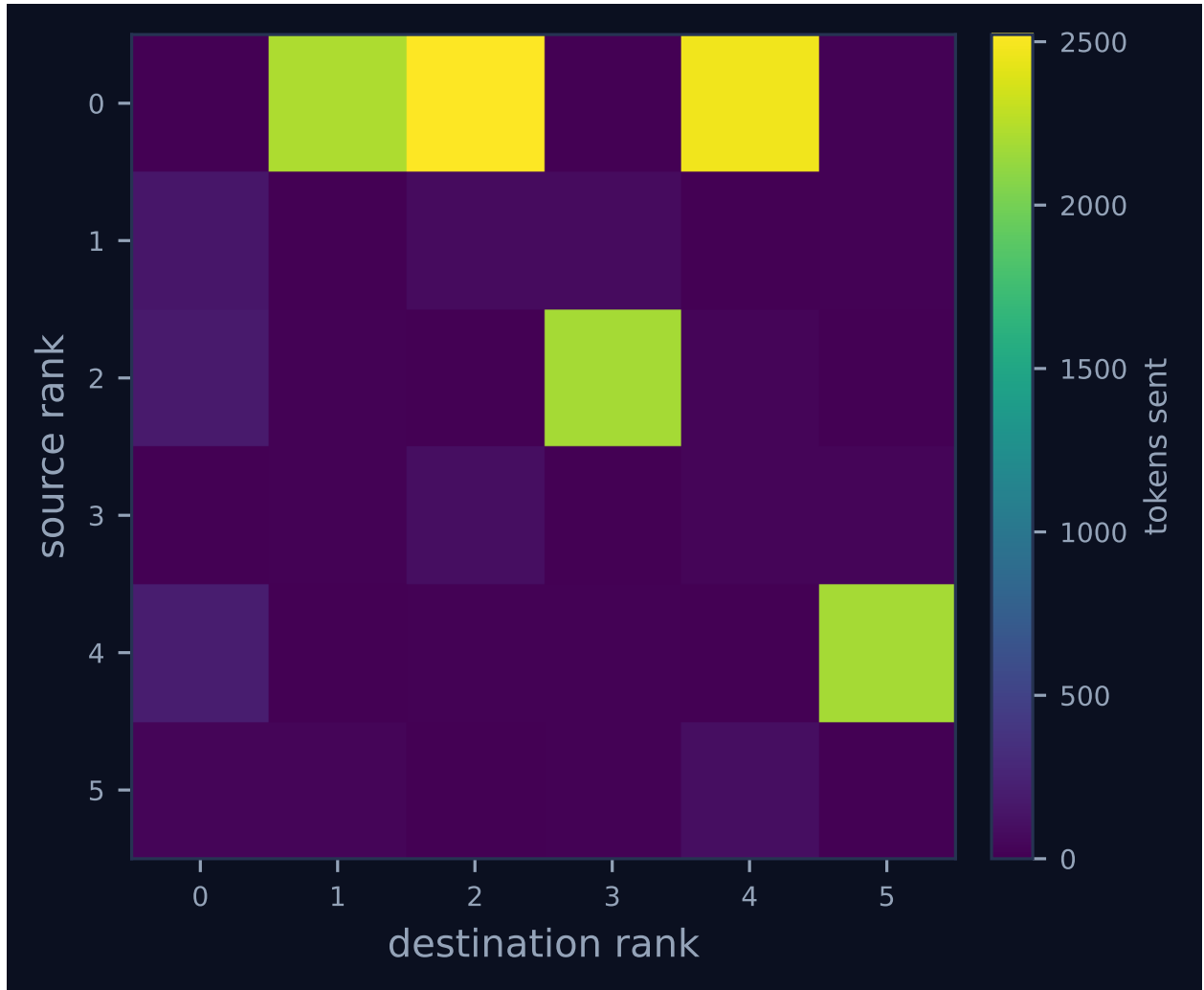


Figure 2: Communication matrix: tokens sent from each rank to each rank. The algorithm’s shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

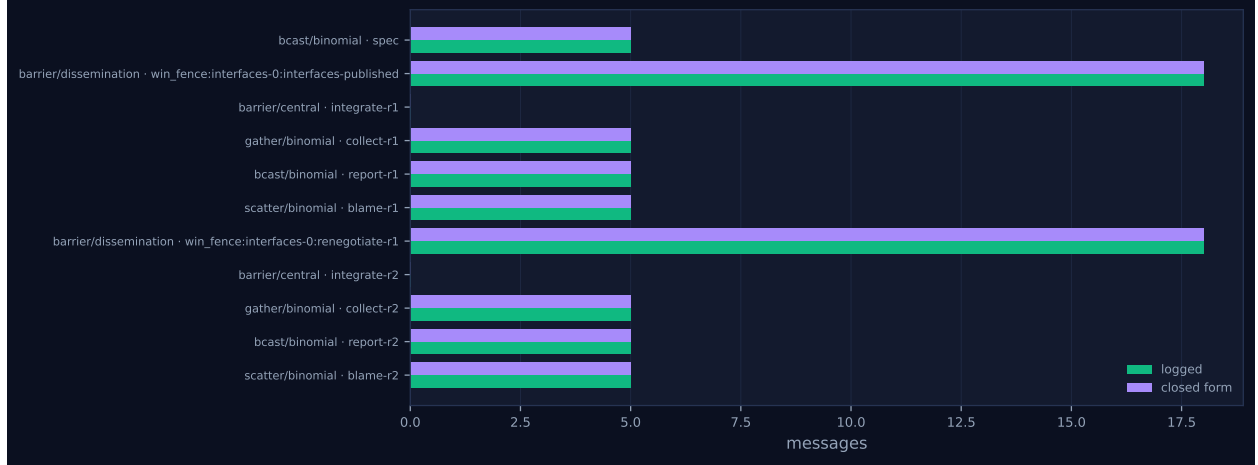


Figure 3: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

agreement marks. The concurrency and rank-profile figures are absent from the generated set for the same reason — with zero busy seconds there is nothing to plot — and the headline table’s achieved parallelism of 0.00 $\times$, parallel efficiency of 0.0 % and idle fraction of 100.0 % are all statements that no rank was ever measurably busy, not statements about scheduling. The [\[note\]](#) that collectives account for 72.5 % of rank-seconds is arithmetically true and interpretively empty: when work time is zero, coordination is the only thing there is to measure.

What is legible in the timeline is phase structure. Reading the log in order, the six lanes move together through `spec` broadcast and `win.create` at $t \approx 0.01$ s; eight `iface` calls and eight `win.puts` to $t = 0.031$ s; the `interfaces-published` fence at $t \approx 0.045$ s, immediately followed on every lane by two `win.sync` events and a burst of `win.gets`; eight `impl*:r1` calls; the `integrate-r1` barrier and the gather to rank 0; the report broadcast and blame scatter; the review exchange between $t = 0.139$ and $t = 0.184$ s; renegotiation under the lock; and then the whole second round to `ft.agree` at $t \approx 0.31$ s. The bulk-synchronous shape is there, compressed into a third of a second.

The one visual detail worth pausing on is in the COLLECTIVES panel of the dashboard rather than in the timeline. Five operations are listed. Four of them — `barrier`, `bcast`, `gather`, `scatter` — carry an algorithm name and a message count. The fifth reads `neighbor_allgather`, 12 invocations, algorithm “—”, max rounds 0, MESSAGES 0. A reader deciding whether the neighbourhood collective earned its place would consult that row, and that row says it moved nothing. Figure 2 disagrees: the review traffic is plainly there as an off-diagonal band, with $0 \rightarrow 1$, $0 \rightarrow 2$, $2 \rightarrow 3$ and $4 \rightarrow 5$ each carrying eight messages. The two panels are drawn from different sources — the collective’s self-report and the fabric’s tagged traffic — and only one of them is right.

7 Interpretation

What is true by construction

The configuration is `-ranks 6 -rounds 2 -executor function`, with shared interfaces, locks and review all enabled, the precise specification, and smoke-scale guards: `barrier_timeout 25 s` and `job_timeout 90 s` against 2,400 s and 14,400 s in the real runs. Everything below follows from that.

Module assignment is $i \bmod p$ over the eight-entry table, so with $p = 6$ two ranks own two modules and four own one: rank 0 has **errors** and **engine**, rank 1 **tokens** and **api**, and ranks 2–5 have **nodes**, **functions**, **parser** and **planner** respectively. The per-rank agent-call counts in the generated table — 7, 7, 4, 4, 4, 4 — are exactly this: one interface call and one implementation call per module per round, plus one review call per rank. This is the only run in the archive with a non-integral modules-per-rank ratio; $p = 1$, $p = 4$ and $p = 8$ all divide evenly.

The build fails in both rounds and that is intended. `synthetic_agent` returns `def stub_<name>(): return None` for every implementation, so the eight-file tree is 24 lines long and `minidb/__init__.py` cannot import `query` from `minidb.api`. The oracle reports `importable: false`, the harness broadcasts the synthetic `IMPORT` record to all six ranks — hence `n_my_failures: 1` for every rank in every round — and `agree` returns `false` twice. A smoke test that went green would exit the loop after round 1 and never reach review or renegotiation at all. Failing is how this run reaches the code it was written to exercise, and it is why a run named “smoke” logs 442 events — more than the eight-rank `vague-sw-p8-vague-shared` at 311, which went green in round 1 and skipped both phases, and within 7% of `real-sw-p8-noshared` at 473.

The RMA traffic follows the same construction: 6 `win.create`, 16 `win.put` (eight slots published, eight renegotiated), 44 `win.get` (14 dependency reads per round, plus a value read and a version read inside each of the eight critical sections), 24 `win.sync` and 8 `win.lock/win.unlock` pairs. The contention report records zero contended locks, zero lock wait, zero stale writes and zero staleness violations, which is expected rather than reassuring: the window is slot-partitioned by ownership and no two ranks ever address the same slot, so the locking path cannot be contended by construction in any configuration of this harness.

The review routing is correct, and this run is why we know

This is the run’s purpose. The mechanism is subtle enough to restate: `review_edges` yields (`author`, `reviewer`) pairs, so under `dist_graph_create` a rank’s `sources` are the authors whose code it reviews and its `destinations` are the reviewers of its own code. Code flows along the edges; a critique has to flow back along them, which requires the transpose. Sending the critique over the forward graph delivers it to the ranks a reviewer sends code to — a set disjoint from the ranks it reviewed.

The trace contains both graphs. Twelve `topo.graph_create` events appear, two per rank, and they are exact transposes of each other. Rank 0 records `sources: [4, 5]`, `destinations: [1, 2]` for the first and `sources: [1, 2]`, `destinations: [4, 5]` for the second; the other five ranks are the corresponding rotations of a degree-2 circulant on six vertices, which is what `review_edges(6, fanout=2)` produces.

The decisive evidence is the traffic, because a topology can be built and then not used. The 95 `msg.send` events carry internal tags, and the two neighbourhood exchanges are `_ampi:nbr_ag:0:8` (the outgoing code, 12 messages, 560 tokens) and `_ampi:nbr_ag:0:9` (the returning reviews, 12 messages, 156 tokens). Grouping each by sender:

sender	code out (review-r1)	review back (reviewback-r1)
0	1, 2	4, 5
1	2, 3	0, 5
2	3, 4	0, 1
3	4, 5	1, 2
4	0, 5	2, 3
5	0, 1	3, 4

Rank 0’s sources are 4 and 5, so rank 0 reviewed the code written by ranks 4 and 5, and rank 0’s reviews went to ranks 4 and 5. The same holds for all six rows: the right column is the left column’s transpose, edge for edge, and every one of the twelve return messages was addressed to a rank whose code its sender had actually received. **This run routed its reviews correctly.** For contrast, the same check on `real-sw-p8-full` finds all sixteen reviewback messages delivered to ranks the sender had never reviewed.

The dating is worth stating because the provenance field misleads. This result was written at 08:29:57 and stamps commit `a803ffd`, which is the *pre-fix* commit, because `provenance()` calls `git rev-parse HEAD` when the result is written rather than reading the working tree. Taken at face value that would say the run executed the buggy code — and the trace plainly shows it did not. The resolution is in the repository: commit `3012e0f`, “Fix review misrouting: a directed collective needs the graph AND its transpose”, landed at 08:31:10, seventy-three seconds after this result was written, and its diff *adds* `results/software/sw-rev-smoke-p6-shared-locks-review-precise-r2.json` alongside the change to `experiments/software.py`. The run was executed against an uncommitted working tree and committed as the fix’s evidence. That is also the only reason there is a run named `sw-rev-smoke` at all.

One half of the fix is *not* verified here, and the distinction matters. The same commit added an ownership filter, so that a reviewer’s cross-module remarks are recorded rather than acted on by the wrong rank. `synthetic_agent` returns `{"target": "peer", "findings": []}` for every review call, so the per-rank stats record `review_findings_mine: 0` and `review_findings_foreign: 0` for all six ranks. With no findings, the filter has nothing to classify. This run establishes that the *addressing* is right; that the *content* reaching an author is about that author’s file is covered by the two unit tests the same commit added to `tests/test_topology.py`, not by this trace.

The collective that could not report its own saving

Both [warning] findings are the same defect seen twice, one per invocation, and the generated list is right to raise them. In this log a `coll.neighbor_allgather` payload is `{"indegree": 2, "outdegree": 2, "label": ..., "wall_s": ...}` and nothing else. There is no `messages_sent` field to read, no `tokens_sent`, no `algorithm` and no `rounds`, so `metrics.json` records `messages: 0`, `tokens: 0`, `algorithm: "?"`, `rounds: 0` for each of the twelve invocations, and the model column is – because there is no closed form for an unnamed algorithm.

The traffic is entirely recoverable, and the analysis tool recovers it: the `msgs` column of the collectives table reads 12 for both rows, derived from the `_ampi:nbr_ag:` tags rather than from the self-report, which is precisely what makes that table a check. What it under-recovers is tokens. `metrics.json` holds `logged_tokens: 560` and `156` for the two invocations, but the LaTeX table’s token column prints the self-reported `tokens` field and therefore reads 0 for both. Messages come from the log and tokens come from the collective in the same row; that inconsistency is a small reporting defect

in the generator, worth fixing because the caption explains the provenance of the message column and says nothing about the token column.

The scale of the omission: 24 of the run’s 95 messages, 25.3 %, and 716 of its 12,720 sent tokens, come from the two exchanges that report zero. And the point is sharper than a missing field, because of which collective it is. A neighbourhood allgather exists for exactly one reason, stated in its own docstring: it costs $\Theta(\text{degree})$ instead of $\Theta(p)$. Here that is 12 messages against the 30 a ring allgather would send at $p = 6$ ($p(p - 1)$) or the 18 recursive doubling would send ($p \lceil \log_2 p \rceil$) — a factor of 2.5 or 1.5 at this size, growing without bound in the first case. A trace that records the degree but not the message count leaves the collective’s entire justification unverifiable from the log, which is the situation this archive was built to avoid.

It is fixed at HEAD, and after this trace. `src/agentmpi/topology.py` at commit `ee21d79` now emits `algorithm="neighbor", messages_sent, tokens_sent` and rounds from both `neighbor_allgather` and `neighbor_alltoall`, with a comment giving the reason above. So this is an expected property of an archived trace of this vintage rather than a live bug, and the correct reading of the two warnings is that the analysis is refusing to trust a self-report — doing its job — not that the collective misbehaved. Note in passing that no `coll.neighbor_alltoall` event appears in this trace at all: the corrected harness uses `neighbor_allgather` in both directions, and `neighbor_alltoall` was what the pre-fix version used for the return path.

Everything else agrees with the cost model

Eleven of the thirteen invocations are checkable and all eleven sent exactly the predicted number of messages: the binomial broadcasts 5 each, the binomial gathers and scatters 5 each, and the dissemination fences 18 each. At $p = 6$ the harness takes the binomial path for gather and scatter, unlike the $p = 4$ smokes, which fall on the `linear` side of the `p <= 4` rule. The `central` barriers report 0 messages and are predicted to send 0, which is correct: that algorithm signals through the fabric’s barrier table rather than by message.

What this run contributes

It is the only archived software run at $p = 6$, the only one with uneven module ownership, and the only one with correct review routing that also has a review phase — the vague-specification pair ran after the fix but `vague-sw-p8-vague-shared` went green in round 1 and skipped review entirely. It is therefore the archive’s only positive evidence that the review topology works as designed, and it exists because someone wrote it to be that evidence. What it cannot contribute is anything about the experiment’s actual question. The tree it builds is 24 lines of stubs, the acceptance score is 0/60 by construction, and the \$0.1492 and 45,501 prompt tokens in the headline table are prices for prompts that were assembled and then answered by a dictionary lookup.

8 Threats to this reading

The executor is a surrogate, so nothing here is about a model. `executors:` `function` $\times 6$. Every conclusion above is about the protocol and the harness. The 0.324s wall time is SQLite writes and event logging; the agent latencies of 0.00s are not fast inference, they are the absence of inference. The cost and token figures are notional: the prompts were really built, and their 45,501

tokens were really counted, but no model ever saw them, so \$0.1492 is what this run *would* have cost and 0.1764\$/kTok out is an artefact of dividing a real prompt bill by 846 stub completion tokens.

Routing is verified at the level of addresses, not meaning. The check above is that message m went from rank a to rank b where b had earlier sent a its code. It does not show that a critique of `parser.py` reached the owner of `parser.py`, because under the surrogate every critique is the empty list. The stronger property is what the two tests added in 3012e0f assert, and a real-agent run with review enabled and a build that fails on the first round would be the trace that demonstrates it end to end. No such run is in the archive.

$p = 6$ is small enough to hide an off-by-one. A degree-2 circulant on six vertices has the property that $\{r+1, r+2\}$ and $\{r-1, r-2\}$ are disjoint, which is what makes the transpose check clean here. It is not obviously robust: at $p = 3$ with fanout 2 the forward and reverse neighbour sets coincide, and the same trace evidence would be consistent with either routing. Any regression test of this property should not be run at $p = 3$ or $p = 4$.

Two derived quantities are degenerate rather than informative. The coordination share of 72.5% and span share of 91.9% divide by rank-seconds and wall clock in a run where work time is identically zero, so they cannot be compared with the same metrics on the real-agent runs; `real-sw-p8-full`’s 41.7% is a fraction of a pool that was really doing something. Likewise the load imbalance of $0.00\times$ means the denominator was zero, not that the six ranks were perfectly balanced — and with two ranks owning two modules each, they were not.

The twelve `ft.agree` events are counted as recovery. `role_counts` reports 12 recovery events against zero trouble events, and the viewer’s legend shows “fault recovery 12” beside FAILURES 0. Nothing was recovered from: both `agree` invocations record `voters: [0..5]` and `excluded: []`. The harness uses `agree` as its decision primitive on the happy path, and the visual vocabulary maps every `ft.*` event to the fault-tolerance palette. This is a taxonomy artefact shared with every software run.

One run, and its purpose biases what it shows. A verification run is executed until it demonstrates the thing it was written to demonstrate. That this one shows correct routing is strong evidence about the code path; it is weak evidence about how often the harness gets directed topologies right in general, and the bug it verifies was found by an agent reading its own prompts rather than by any test or metric in this system. Message counts, degrees and round counts were all correct while every review went to the wrong rank, which is the most transferable lesson available here: a directed collective can be perfectly well-formed and completely wrong, and no quantity in this trace would have caught it.